Table of Contents

A concise, presentation-ready guide to the project

NO.	SECTION	WHAT YOU WILL FIND
01	Abstract	Project overview and outcome
02	Introduction	Purpose and user journey
03	Problem Statement & Objectives	Need, goals and expected value
04	Scope & Technology Stack	What is included and tools used
05	Workflow & System Architecture	How the system works end-to-end
06	Core Features & Implementation	AI generation, editing and export
07	Milestones, Testing & Validation	Delivery plan and quality checks
08	Challenges & Solutions	Engineering decisions and reliability
09	Limitations, Future Scope & Conclusion	Responsible use and next steps
10	References	Key technical sources
11	Application Screenshots	Visual walkthrough of the running app

Abstract

Project summary

LegalEase is a browser-based application that uses Generative AI to create structured first drafts of common legal documents. Instead of starting with a blank page, users select a document type and provide the parties, key terms and effective date. The application sends these details to Google Gemini and displays the returned draft in an editable preview.

The project combines a Streamlit interface with a FastAPI service layer and a dedicated AI and formatting core. Users can revise generated text, add optional jurisdiction details, apply company branding and export the final draft in TXT, DOCX or PDF format. The Word and PDF outputs support a more professional presentation through logos, headings, terms formatting and footers.

A central design choice is separation of concerns: the frontend manages interaction, the backend validates and routes requests, the AI module builds prompts and handles generation failures, and the export module prepares downloadable files. The application also includes mock mode, which supports demonstrations and automated tests when a Gemini API key is unavailable.

OUTCOME LegalEase streamlines routine first-draft preparation and formatting. It is a drafting assistant—not a legal-advice system or a guarantee of enforceability.

Keywords

Generative AI • Legal document drafting • Google Gemini • Streamlit • FastAPI • Document automation • DOCX/PDF export

Introduction

Turning simple details into a structured draft

Written agreements are common in employment, freelance work, confidentiality arrangements and property rental. Preparing them from scratch requires attention to structure, formal wording and presentation. Reusing an old document can also leave outdated or irrelevant clauses in place.

LegalEase offers a guided drafting flow. A user selects a document category, enters the parties, adds terms separated by semicolons, and supplies an effective date. An optional jurisdiction can provide additional context. Gemini then produces a multi-section draft, which the user can inspect and edit before downloading.

How the user journey feels

1. Describe the agreement using a small set of form fields.
2. Generate a structured draft through the AI core.
3. Review the preview and correct details or placeholders.
4. Apply optional company branding and export the document.

Design principles

PRINCIPLE	HOW IT APPEARS IN THE PROJECT
Accessible	A browser-based Streamlit interface; no local installation for the live app.
Editable	Users can revise the generated text before any download.
Portable	TXT, DOCX and PDF formats support different workflows.
Maintainable	Frontend, API, AI generation and formatting are separated.
Transparent	The tool states that output is a draft and may require legal review.

Problem Statement & Objectives

Why the project is useful

Problem statement

Individuals, freelancers and small businesses often need routine agreements but may not have a reusable, well-structured template or immediate access to professional drafting support. Three recurring obstacles are uncertainty about the clauses a document should contain, difficulty converting everyday terms into formal wording, and the manual effort needed to create a polished final file.

CHALLENGE	EFFECT ON THE USER	LEGALEASE RESPONSE
Structure	Important sections or signature details may be missed.	Generates a multi-section draft with numbered clauses.
Wording	Informal notes are difficult to express consistently.	Uses a structured prompt to produce formal first-draft language.
Presentation	Formatting, branding and page details take time.	Exports formatted Word and PDF files as well as plain text.

Project objectives

- Generate a structured legal first draft from four core inputs.
- Adapt the generated clauses to the selected document type, parties and supplied terms.
- Provide a preview and editing step before export.
- Support TXT, DOCX and PDF downloads with optional branding.
- Keep the interface, API, AI logic and formatting modules independently maintainable.
- Deploy a public version that can be opened in a browser.

IMPORTANT BOUNDARY LegalEase does not verify legal validity or replace a qualified lawyer. Users must complete placeholders and arrange professional review for important documents.

Scope & Technology Stack

What the application delivers

Scope

IN SCOPE	OUT OF SCOPE
Drafting contracts, NDAs, leases, offer letters and general agreements.	Legal advice or verification of enforceability.
Editable preview and TXT/DOCX/PDF export.	Digital signatures or e-signing workflows.
Optional jurisdiction and company branding.	User login, document history and saved accounts.
Local, Docker and Streamlit Cloud run modes.	Analysis of existing contracts uploaded by a user.

Technology stack

LAYER / TOOL	RESPONSIBILITY
Streamlit	Form inputs, preview, editing, sidebar settings and download buttons.
FastAPI + Uvicorn	REST endpoints, request validation, CORS and health checks.
Google Gemini + google-genai	Prompt-based document generation and model access.
python-docx, fpdf2, Pillow	Word/PDF creation, logos, terms formatting and footers.
python-dotenv + config.py	Configuration from environment variables, .env or Streamlit secrets.
pytest	Automated tests, including mock-mode generation and export checks.
Docker / Procfile	Packaging and optional separate backend hosting.

DEPLOYMENT The documented live version runs on Streamlit Community Cloud in standalone mode. API mode is also supported for a separately hosted FastAPI backend.

Workflow & System Architecture

A clear path from user input to downloadable file

1 • STREAMLIT FRONTEND
Collects document type, parties, terms and effective date; displays preview and editing controls.

2 • FASTAPI BACKEND (API MODE)
Validates the request model and routes generation/export requests.

3 • GEMINI AI CORE
Builds a structured prompt, calls Gemini, retries/falls back when possible, or uses mock mode.

4 • EXPORT LAYER
Sanitizes text and creates TXT, DOCX or PDF files with formatting and optional branding.

Request lifecycle

5. The user completes the form and clicks Generate Document.
6. The frontend sends a JSON request; FastAPI validates the required fields in API mode.
7. The AI core assembles a prompt and requests a response from Gemini.
8. The generated draft returns to the interface and appears in a styled preview.
9. The user edits the text and selects TXT, DOCX or PDF for download.

RUN MODES API mode sends requests to a running FastAPI service. Standalone mode lets Streamlit call the AI core directly, which suits a single-process cloud deployment.

Core Features & Implementation

How the design becomes a working application

FEATURE	IMPLEMENTATION DETAIL	USER VALUE
Guided input	Document type, parties, terms, date and optional jurisdiction.	A focused form reduces blank-page friction.
AI generation	GeminiDocumentGenerator constructs the prompt and calls the model.	Produces a structured first draft tailored to supplied details.
Editable preview	Generated text is kept in session state and exposed in an edit area.	Users can fix placeholders and wording before export.
Branding	Sidebar accepts a logo, company name and footer email.	Documents can look consistent with a business identity.
Multi-format export	Separate formatter functions create TXT, DOCX and PDF.	One draft supports multiple downstream uses.
Fallback and mock mode	Retries/model fallback and a sample document when configured.	Demonstration and testing can continue when AI access is unavailable.

Export formats

- TXT — plain UTF-8 text containing the edited draft.
- DOCX — Word document with logo, Times New Roman styling, terms table and page-numbered footer.
- PDF — formatted pages with title/logo header, section styling and footer details.

Key backend routes

METHOD / ROUTE	PURPOSE
GET /	Welcome message and service check.
POST /generate	Accepts required fields and returns generated document text.
POST /export/{fmt}	Formats supplied text as TXT, DOCX or PDF.
GET /health	Reports service status and mock-mode state.
GET /document-types	Returns available document categories.

Milestones, Testing & Validation

From setup to demonstrable deployment

MILESTONE	FOCUS	DELIVERED CAPABILITY
1. Model & architecture	Select model and define layers.	Gemini integration plan, configuration and project structure.
2. Core functionality	Generate, preview, edit and export.	AI generator and TXT/DOCX/PDF formatting.
3. API integration	Create request model and endpoints.	FastAPI routes including generation, export, health and document types.
4. Frontend	Build the interactive user interface.	Streamlit form, dark theme, example loader and branding controls.
5. Deployment	Package and publish the application.	Run scripts, Docker definitions and public cloud deployment.

Validation approach

- Automated pytest suite runs in mock mode without requiring an API key.
- Manual API checks can be performed through FastAPI's /docs interface and command-line requests.
- Functional checks cover multiple document types, long term lists, special characters, editing before export and branding.
- Failure-path checks cover missing API keys, invalid credentials and rate-limit responses.

Expected quality checks

TEST AREA	EXPECTED RESULT
Different document types	The draft structure reflects the selected agreement category.
Edit then export	All downloaded formats contain the user's revised text.
Special characters	Text remains readable in TXT, DOCX and PDF.
Missing key / mock mode	A sample document keeps the interface testable.
Branding	Logo and footer details appear in supported formatted exports.

Challenges, Limitations & Future Scope

Reliability with responsible use

Challenges and solutions

CHALLENGE	RESPONSE
Model retirement / SDK change	Moved to the current configurable Gemini model alias and google-genai SDK.
API failures and rate limits	Added retries, model fallback and clearer messages for common errors.
No API key during testing	Added mock mode so the UI and export flow can be tested independently.
Plain text needed polished output	Built a separate export layer for logos, headings, terms tables and footers.
Cloud deployment runs one process	Provided standalone mode while retaining API mode for separate hosting.

Limitations and responsible use

- The generated text is not legal advice and does not guarantee enforceability.
- Bracketed placeholders may remain and must be completed or corrected.
- Outputs can vary between runs and may include clauses that do not fit the situation.
- Generation depends on an external AI service and its availability and quotas.
- Users should avoid submitting confidential information they would not share with a third-party AI service.

Future enhancements

Potential extensions include reviewed jurisdiction-specific templates, contract analysis of uploaded files, multilingual drafting, user accounts and document history, e-signature integration, legal-reference integration and an automated benchmark for draft quality.

Conclusion

LegalEase demonstrates how Generative AI can reduce repetitive effort in preparing and formatting a legal first draft. Its editable workflow, multiple export formats, modular architecture and mock-mode testing make it practical to demonstrate and extend. Its value is strongest when it accelerates preparation while leaving final review and legal judgment to people.

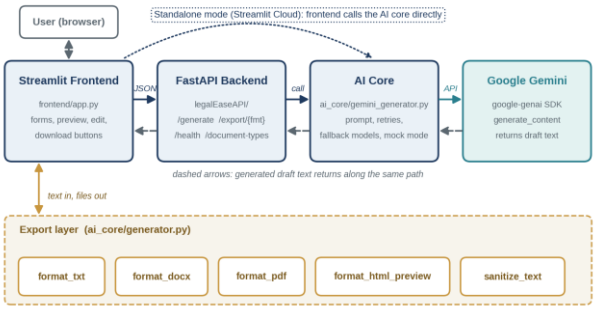
References

- FastAPI documentation — fastapi.tiangolo.com
- Streamlit documentation — docs.streamlit.io
- Google Gemini API documentation — ai.google.dev/gemini-api/docs
- python-docx — python-docx.readthedocs.io
- fpdf2 — py-pdf.github.io/fpdf2/
- pytest — docs.pytest.org
- LegalEase project repository — github.com/tnsridharan10/LegalEase

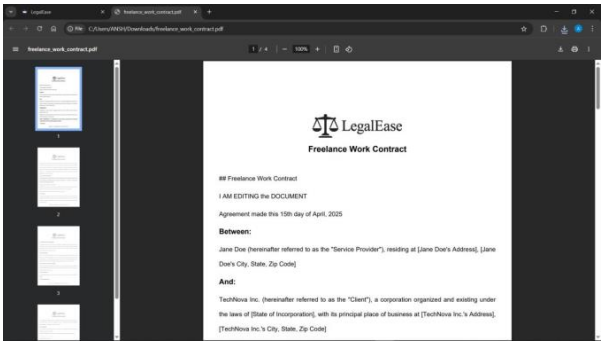
Application Screenshots

Visual walkthrough of LegalEase

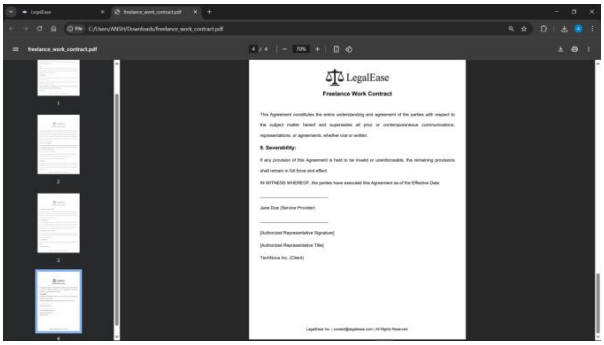
The images below are extracted from the original project document and arranged as a compact application gallery. They illustrate the form, generated preview and export experience.



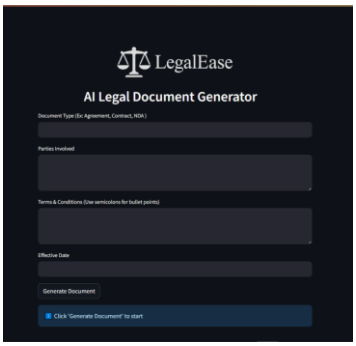
Application interface / input form



Generated document preview



Editing and export controls



Formatted document output

Live application: https://legalease-vp5dqvuzajm864fq9gqqha.streamlit.app/